

master

Go to file

Add file

Code

	fifteenhex sync with local state ...	on Jan 1 94
	br2autosshkey @ 6a5780c	Bits 2 years ago
	br2chenxing @ 5925140	Bumps 10 months ago
	br2dgppkg @ b957386	- Bump br2dgppkg last year
	br2directfb2 @ a7a5fbf	sync with local state 3 months ago
	br2idosom2d01	sync with local state 3 months ago
	br2secretsauce @ f5c53fc	sync with local state 3 months ago
	buildroot @ 44be514	sync with local state 3 months ago
	buildroot_rescue @ 44be514	sync with local state 3 months ago
	.gitignore	Build a ubi image with u-boot in it for direct flashing with SN... 2 years ago
	.gitmodules	- Fix url for dfb2 submodule last year
	.woodpecker.yml	CI: push tagged builds to github as releases last year
	Dockerfile	ci: Add mtd-utils back to dockerfile last year
	LICENSE	Initial commit 2 years ago
	Makefile	- Add dfb2 last year
	README.md	README: Well you can flash the ubi data via the ISP now but... last year
	sbc02d06_jumpers_flashing.jpg	Add some info about the sbc2d06 [CI SKIP] 2 years ago
	sbc02d06_jumpers_usb_serial.jpg	Add some info about the sbc2d06 [CI SKIP] 2 years ago

About

Buildroot for ido-som2d01 (and similar SSD20xD) based boards.

#linux #buildroot #sigmastar #ssd202d #ssd201

- Readme
- GPL-3.0 license
- 3 stars
- 3 watching
- 4 forks
- Report repository

Releases 14

build_20220404_00 Latest on Apr 4, 2022

+ 13 releases

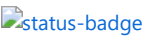
Packages

No packages published

Languages

- Makefile 80.9%
- Dockerfile 10.9%
- JavaScript 4.7%
- Shell 3.5%

buildroot_ido-som2d01



Buildroot for ido-som2d01 based boards.

Wha?

This is a buildroot setup for the various ido-som2d01 based boards.

- The som2d01 with an SSD201 or SSD202D
- The [sbc2d06](#)
- The [sbc2d70](#)

This is not a all singing all dancing OS image for these things. This is meant to be something that builds a complete working set of binaries that you can work from.

Flashing a blank module

- You need to flash at least the CIS blob, the IPL blob and the u-boot SPL. Right now you need to do this with the vendor ISP tool. Flashrom can drive the i2c interface to write SPI NOR but it cannot do SPI NAND yet so there is no alternative right now.
 - See [here](#) for information on this.
 - Note that the u-boot SPL is currently called `idosom2d01-ip1`. At some point the u-boot SPL will replace the vendor IPL blob.

Loading the UBI partition data with the ISP tool is very very slow so you don't to do that. The u-boot SPL is capable of loading the main u-boot binary over serial. The first time you boot you need to send u-boot with ymodem. This should be a lot faster than trying to flash the UBI partition via the ISP tool.

- Now we need to format the part of the flash we'll use for the UBI partitions. `idosom2d01-kernel-rescue.fit` is a micro system with a kernel and just enough userland to setup the flash.
- If you board has ethernet you can put `idosom2d01-kernel-rescue.fit` onto a TFTP server somewhere and load it via ethernet.

```
setenv loadaddr 0x22000000; dhcp 192.168.3.235:idosom2d01-kernel-rescue.fit; bootm
```

- If you don't have ethernet you can load over serial with ymodem. This will be insanely slow though so only do this as last resort.

```
setenv loadaddr 0x22000000; loady; bootm
```

Depending on your serial interface you might be able to increase the baud rate used during the transfer.

If you are really lucky something like this might work:

```
setenv loadaddr 0x22000000; loady ${loadaddr} 230400; bootm
```

or

```
setenv loadaddr 0x22000000; loady ${loadaddr} 460800; bootm
```

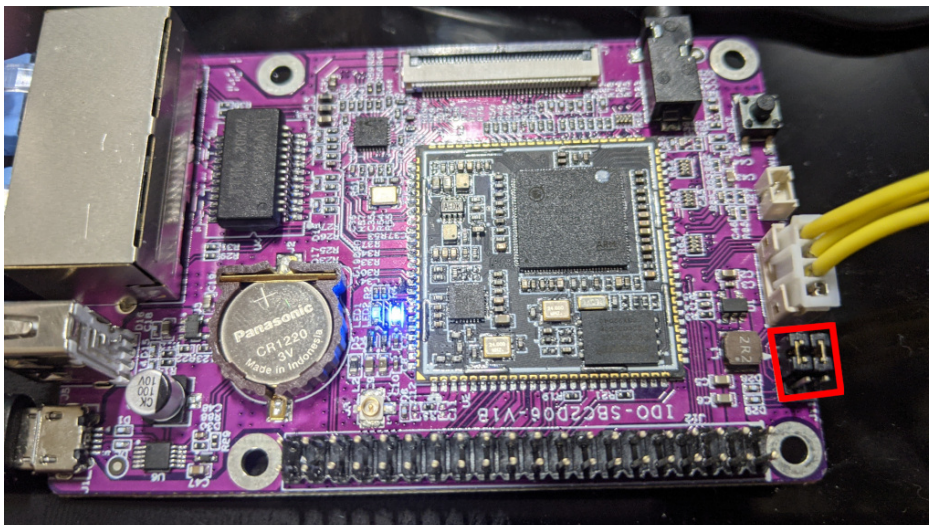
Eitherway, you should now be booted in a Linux environment where you can finish off flashing the required images.

Note for SSD202D modules you must use `bootm ${loadaddr}#ssd202d-som2d01` to use the SSD202D device tree otherwise the kernel locks up at boot.

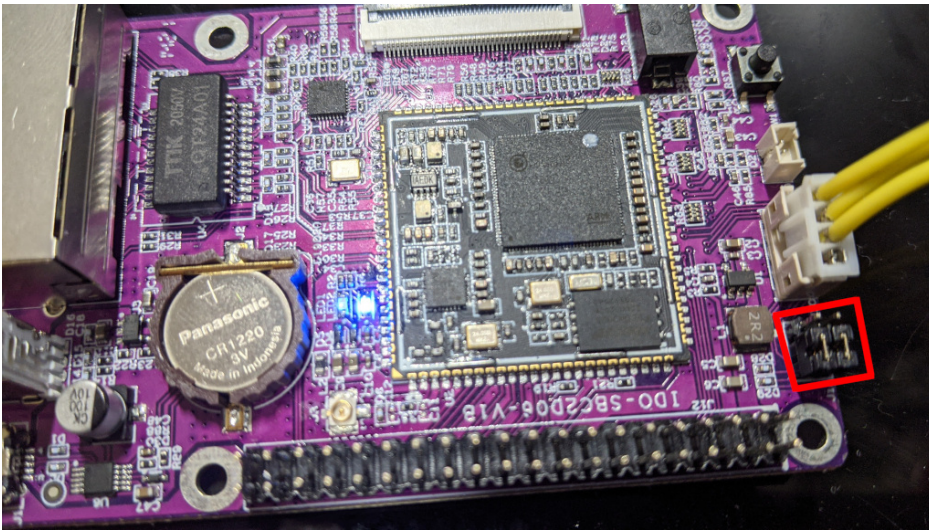
Notes for SBC2D06

To write the GCIS, IPL and u-boot SPL on the module on the SBC2D06 you need to move the jumpers.

This is the position for using the usb port for serial:



This is the position for using the ISP connector for flashing.



You need to move the jumpers into the flashing position when using the the ISP tool and back again when you need to access the serial console.

Using the rescue image to complete flashing

- First we need to create the UBI partitions

```
ubiformat /dev/mtd1
ubiattach -m 1
ubimkvol /dev/ubi0 -N uboot -s 1MiB -t static
ubimkvol /dev/ubi0 -N env -s 256KiB
ubimkvol /dev/ubi0 -N kernel -s 16MiB
ubimkvol /dev/ubi0 -N rescue -s 16MiB
ubimkvol /dev/ubi0 -N rootfs -m
```

- Now we should pull the rescue image and load it into the rescue partition. If something goes wrong once we have the rescue image in the flash it's a lot easier to progress. Especially if you are using serial.

**** tftp instructions here ****

- For serial run `rz` and then send `idosom2d01-kernel-rescue.fit`.
- Now run:

```
ubiupdatevol /dev/ubi0_3 idosom2d01-kernel-rescue.fit
```

- Next we should pull the u-boot image and flash it.

- For serial run `rz` and then send `idosom2d01-u-boot.img`.
- Now run:

```
ubiupdatevol /dev/ubi0_0 idosom2d01-u-boot.img
```

- At this point your module should be able to boot into u-boot directly without needing serial and you should be able to boot back into the rescue setup by running:

```
setenv loadaddr 0x22000000; ubi readvol ${loadaddr} rescue 0x800000; bootm
```

- Now lets get the kernel and rootfs images

- For serial run `rz` and then send `idosom2d01-kernel.fit` and `idosom2d01-rootfs.squashfs`
- For tftp something like:

```
tftp -b 4096 -g -r idosom2d01-kernel.fit 192.168.3.235
tftp -b 4096 -g -r idosom2d01-rootfs.squashfs 192.168.3.235
```

- Now run:

```
ubiupdatevol /dev/ubi0_2 idosom2d01-kernel.fit
ubiupdatevol /dev/ubi0_4 idosom2d01-rootfs.squashfs
```

- Now everything is flashed, you should be able to boot the normal kernel with a more feature rich rootfs. The supplied rootfs is a read only squashfs. If you want to use ubifs for a read/write filesystem you can do that but it won't be detailed here.

```
setenv bootargs console=ttyS0,115200 ubi.mtd=1 ubi.block=0,4 root=/dev/ubiblock0_4
clk_ignore_unused
setenv bootcmd 'setenv loadaddr 0x22000000; ubi readvol ${loadaddr} kernel 0x800000;
bootm ${loadaddr}#<your board type>'
```

- Have fun!

Appendix: Creating ubi layout in u-boot

```
ubi createvol uboot 0x100000 static
ubi createvol env 0x40000
ubi createvol kernel 0x1000000
ubi createvol rescue 0x1000000
ubi createvol rootfs -
```